



UNIVERSITÀ DEGLI STUDI DI SALERNO

Dipartimento di Informatica

Corso di Laurea Magistrale in Informatica

AI-Driven Firewall Evasion: Sviluppo di un agente autonomo basato su LLM per l'elusione adattiva di controlli di rete.

PROFESSORI

Arcangelo Castiglione

STUDENTI

Vittorio Guida

Matricola: 0522502008

Anno Accademico 2025-2026

Abstract

Questa tesina presenta *AI-Driven Firewall Evasion System*, un Proof of Concept che indaga se un Large Language Model, inserito come agente decisionale in un ciclo iterativo di Trial & Error, sia in grado di automatizzare il ragionamento tipicamente riservato a un penetration tester nell'elusione di un firewall. Il sistema, sviluppato in Python e containerizzato tramite Docker, mette in comunicazione un container attaccante, dotato di un motore di mutazione dei pacchetti basato su Scapy, con un container target protetto da un ruleset Iptables configurato ad-hoc. A ogni iterazione, un motore basato sulle API di Google Gemini analizza la baseline del traffico legittimo e la cronologia dei tentativi falliti per proporre la mutazione di pacchetto più promettente, in un ciclo di osservazione, ipotesi e correzione.

La sperimentazione, condotta a confronto con un motore Mock privo di ragionamento, mostra che la sola capacità del modello linguistico non è sufficiente a garantire una convergenza efficiente: con un prompt privo di un'istruzione esplicita di cambio strategia, il sistema non ha mai raggiunto il successo in oltre 40-50 tentativi; con un prompt opportunamente raffinato, il successo è stato raggiunto in 6 tentativi e riproposto al primo tentativo in una sessione successiva grazie alla persistenza del log. Un ulteriore risultato rilevante, non previsto in fase di progettazione, riguarda le restrizioni di sicurezza applicate dal fornitore del modello: solo la variante più leggera della famiglia Gemini si è dimostrata utilizzabile, a fronte di un compromesso tra capacità di ragionamento e conformità alle politiche di sicurezza del provider. Il lavoro discute infine le implicazioni etiche legate all'uso offensivo dell'Intelligenza Artificiale e delinea le direzioni di sviluppo per un possibile approfondimento in sede di tesi di laurea.

Indice

Elenco delle Tabelle	iii
1 Introduzione	1
1.1 Contesto e motivazioni	1
1.2 Obiettivi della tesina	2
1.3 Struttura del documento	3
2 Background e Lavori Correlati	4
2.1 Firewall e tecniche di evasione	4
2.2 LLM come agenti autonomi per la sicurezza offensiva	5
2.3 Adversarial AI e retroazione iterativa	6
3 Architettura del Sistema	7
3.1 Dal progetto iniziale all'implementazione	7
3.2 Overview generale	8
3.3 Ambiente Dockerizzato e isolamento di rete	9
3.4 Traffic Analyzer: profilazione della baseline	9
3.5 Mutation Engine: mutazione dei pacchetti tramite Scapy	9
3.6 LLM Engine: motore reale e motore Mock	10
3.7 Success Feedback Analyzer e orchestratore	11

4	Metodologia Sperimentale	13
4.1	Setup del testbed	13
4.2	Definizione degli scenari di test	15
4.3	Metriche di valutazione	16
4.4	Criteri di riproducibilità	17
5	Risultati e Analisi	19
5.1	Risultati sperimentali	19
5.2	Analisi delle strategie di evasione scoperte dall'agente	21
5.3	Limiti riscontrati	22
5.3.1	Restrizioni di sicurezza a livello di modello e loro impatto sul sistema	22
5.3.2	Altri limiti dello studio	23
5.4	Discussione critica dei risultati	24
6	Considerazioni Etiche e Implicazioni di Sicurezza	26
6.1	Uso responsabile della ricerca offensiva	26
6.2	Implicazioni difensive	27
6.3	Rischi di dual-use e mitigazioni	27
7	Conclusioni e Sviluppi Futuri	30
7.1	Sintesi dei contributi	30
7.2	Sviluppi futuri	31
7.3	Riflessione conclusiva	33
	Bibliografia	34

Elenco delle tabelle

4.1	Scenari sperimentali definiti	15
5.1	Sintesi dei risultati sperimentali per scenario	21

CAPITOLO 1

Introduzione

1.1 Contesto e motivazioni

I sistemi di firewall e i meccanismi di packet filtering rappresentano da decenni la prima linea di difesa perimetrale nelle infrastrutture di rete, con il compito di applicare politiche di sicurezza sul traffico in transito sulla base di regole statiche o stateful. Fin dalla fine degli anni '90, tuttavia, la letteratura scientifica ha dimostrato che tali sistemi possono essere elusi tramite tecniche di manipolazione dei pacchetti di rete progettate per sfruttare le discrepanze di interpretazione tra il dispositivo di difesa e l'host bersaglio [1].

Tradizionalmente, l'individuazione di queste tecniche di evasione ha richiesto un'attività manuale e altamente specialistica da parte di un penetration tester. Negli ultimi anni, però, l'avvento dei Large Language Model (LLM) ha aperto la strada a un nuovo paradigma di automazione dell'attività offensiva: un LLM può orchestrare autonomamente sotto-task di un penetration test [2], mentre analisi più recenti ne evidenziano sia il potenziale sia i limiti attuali, in particolare la tendenza alle allucinazioni [3].

Il presente lavoro si colloca esattamente in questo punto di intersezione, combinando le tecniche di evasione tradizionali con le capacità decisionali degli agenti AI

autonomi. Il progetto nasce dall'esigenza di verificare sperimentalmente se un LLM di uso commerciale (Google Gemini), inserito in un ciclo di retroazione automatizzato, sia in grado di sostituire il ragionamento umano nella fase di individuazione e adattamento di una strategia di evasione contro un firewall Iptables, basandosi sull'osservazione del traffico di rete legittimo (*baseline*) e sulla cronologia dei tentativi falliti.

Il sistema è stato interamente sviluppato in Python e containerizzato tramite Docker, con un container *attaccante* (dotato dei privilegi di rete necessari alla manipolazione di pacchetti raw tramite Scapy [4]) che dialoga con un container *target* protetto da regole Iptables. L'intelligenza del sistema risiede in un motore di analisi (`llm_engine.py`) che interroga le API di Google Gemini [5] per dedurre, a ogni iterazione, la mutazione di pacchetto più promettente da applicare.

1.2 Obiettivi della tesina

Alla luce del contesto delineato, la presente tesina si pone i seguenti obiettivi:

- Fornire un inquadramento essenziale delle tecniche di firewall/IDS evasion e delle applicazioni dei LLM alla sicurezza offensiva, posizionando il progetto rispetto allo stato dell'arte;
- Descrivere in dettaglio l'architettura del sistema *AI-Driven Firewall Evasion System*, analizzando il ruolo di ciascun componente all'interno del ciclo di Trial & Error;
- Illustrare la metodologia sperimentale adottata per validare il sistema in un ambiente Dockerizzato controllato;
- Discutere criticamente i risultati ottenuti, evidenziando punti di forza e limiti dell'approccio;
- Analizzare le implicazioni etiche del lavoro e le possibili ricadute positive in ottica difensiva.

1.3 Struttura del documento

Il resto della tesina è organizzato come segue. Il Capitolo 2 fornisce un inquadramento teorico essenziale su firewall, tecniche di evasione e LLM agentici, posizionando il progetto rispetto ai lavori correlati. Il Capitolo 3 descrive nel dettaglio l'architettura del sistema sviluppato. Il Capitolo 4 illustra la metodologia sperimentale adottata. Il Capitolo 5 presenta e discute i risultati ottenuti. Il Capitolo 6 affronta le considerazioni etiche legate al progetto. Infine, il Capitolo 7 riassume i contributi del lavoro e delinea possibili sviluppi futuri.

Background e Lavori Correlati

2.1 Firewall e tecniche di evasione

Un firewall applica una politica di sicurezza al traffico di rete decidendo se inoltrare, scartare o rifiutare ciascun pacchetto. I firewall, come Iptables (l'interfaccia storica del framework Netfilter di Linux qui utilizzata come target), adottano un approccio *stateful*, mantenendo una tabella delle connessioni attive e valutando ogni pacchetto anche in funzione della coerenza con lo stato atteso della connessione. Rilevante ai fini di questo lavoro è la distinzione tra le azioni DROP (scarto silenzioso) e REJECT (rifiuto esplicito, ad esempio tramite un pacchetto TCP RST): come si vedrà nel Capitolo 3, il sistema sviluppato si basa esattamente su questa distinzione per inferire lo stato del firewall a ogni iterazione.

Il principio teorico alla base di ogni tecnica di evasione è stato formalizzato da Ptacek e Newsham [1], che hanno introdotto i concetti di *insertion* ed *evasion*: entrambi derivano da un'ambiguità di interpretazione tra lo stack di rete del dispositivo di controllo e quello dell'host finale. Da questo principio derivano tecniche ancora oggi rilevanti manipolazione dei flag TCP, alterazione del TTL, frammentazione irregolare, offuscamento a livello applicativo storicamente implementate da strumenti come Nmap [6] e fragrouter [7] e oggi più spesso ricostruite ad-hoc tramite librerie di basso

livello come Scapy [4]. Tutte queste tecniche condividono un vincolo strutturale: devono preservare la funzionalità della comunicazione pur deviando a sufficienza dal comportamento standard da confondere il dispositivo di controllo, un vincolo che guida direttamente la progettazione del motore di mutazione descritto nel Capitolo 3.

2.2 LLM come agenti autonomi per la sicurezza offensiva

L'idea di affidare a un LLM il ruolo di motore decisionale in un ciclo iterativo di attacco si fonda su paradigmi di prompting come il Chain-of-Thought [8] e ReAct [9], che permettono al modello di esplicitare un ragionamento intermedio e aggiornare il proprio piano d'azione sulla base del feedback ricevuto. Diversi framework recenti hanno applicato questi principi al penetration testing: PentestGPT [2] ha dimostrato che un LLM può gestire sotto-task di un test reale con un incremento del 228,6% nel tasso di completamento rispetto a un utilizzo diretto del modello; VulnBot [10] ha esteso l'approccio a un'architettura multi-agente che replica i ruoli di un team umano; xOffense [11] ha mostrato che anche un LLM open-source di dimensioni intermedie, opportunamente specializzato, può competere con questi sistemi riducendone i costi. Un'analisi critica di Happe [3] evidenzia però limiti tuttora aperti allucinazioni, perdita di coerenza su cicli prolungati particolarmente rilevanti per il meccanismo di gestione degli errori descritto nel Capitolo 3.

Tutti questi lavori, tuttavia, affrontano il penetration testing nella sua interezza (ricognizione, scansione, exploitation), trattando l'evasione di un firewall solo come ostacolo incidentale da aggirare con strumenti esterni preesistenti. Il progetto qui descritto adotta invece un ambito volutamente verticale: l'unico compito dell'agente è l'elusione del dispositivo di controllo tramite mutazione di pacchetti raw, trattata come oggetto di ragionamento diretto da parte del modello.

2.3 Adversarial AI e retroazione iterativa

Un secondo filone rilevante è quello dell'adversarial machine learning applicato ai sistemi di detection basati su modelli statistici. Alhajjar et al. [12] offrono una rassegna delle tecniche con cui un attaccante può generare traffico perturbato capace di ingannare un Network Intrusion Detection System (NIDS); Usama et al. [13] e Zhang et al. [14] mostrano applicazioni concrete di questo principio tramite reti generative avversarie. Questi lavori condividono con il progetto qui descritto il meccanismo di retroazione iterativa (tentativo, osservazione dell'esito, aggiustamento della strategia), ma se ne distinguono per il bersaglio: un modello di classificazione statistico nei lavori citati, contro un insieme di regole esplicite e deterministiche (Iptables) nel sistema qui sviluppato, una differenza che cambia sostanzialmente la natura del problema affrontato dall'agente, spostandolo dalla ricerca di una perturbazione impercettibile rispetto a una superficie di decisione appresa, alla deduzione per tentativi di quali specifici campi di intestazione siano effettivamente ispezionati da una regola statica.

Architettura del Sistema

3.1 Dal progetto iniziale all'implementazione

La progettazione del sistema è partita da un documento di design (*AI-Driven Dynamic Protocol Mutation for Firewall Evasion*) che delineava un'architettura a cinque componenti: Traffic Analysis Engine, Protocol Mutation Generator, LLM Evasion Strategy Engine, Traffic Emitter Module, Success Feedback Analyzer pensata per operare su una gamma ampia di protocolli (TCP/UDP, ma anche header HTTP e segmentazione dei record TLS) contro un generico firewall, con data store esterni dedicati (Elasticsearch per i profili di baseline, Redis per il log delle risposte del firewall) e un ciclo di apprendimento per rinforzo basato su Q-learning o policy gradient.

L'implementazione effettiva, qui descritta, ha mantenuto la suddivisione concettuale in cinque responsabilità ma ne ha semplificato deliberatamente la realizzazione tecnica, a vantaggio della riproducibilità e dell'isolamento dell'ambiente di test: i data store esterni sono stati sostituiti da semplici file strutturati in formato JSON (`baseline.json` ed `evasion_log.json`), il bersaglio è stato ristretto a un firewall Iptables configurato ad-hoc su un container dedicato e il meccanismo di apprendimento per rinforzo esplicito è stato sostituito da un ragionamento diretto

dell’LLM sulla cronologia dei fallimenti, secondo un paradigma più vicino al Chain-of-Thought [8] che al reinforcement learning classico. Questa scelta, meno ambiziosa in termini di infrastruttura ma più controllabile sperimentalmente, è coerente con l’obiettivo della tesina: non costruire un prodotto pronto per la produzione, ma isolare e verificare l’ipotesi centrale del lavoro, ovvero che un LLM possa sostituire il ragionamento umano nel ciclo di evasione.

3.2 Overview generale

Il sistema realizzato, denominato *AI-Driven Firewall Evasion System* [15], è composto da cinque moduli software che cooperano all’interno di un ciclo iterativo di tipo osserva-ragiona-agisci-osserva:

- **Traffic Analyzer** (`traffic_analyzer.py`): profila il traffico di rete legittimo;
- **Mutation Engine** (`mutation_engine.py`): altera la struttura dei pacchetti raw tramite Scapy;
- **LLM Engine** (`llm_engine.py` / `llm_engine_mock.py`): decide la prossima mutazione da applicare;
- **Success Feedback Analyzer** (`successFeedbackAnalyzer.py`): valuta l’esito di ogni tentativo;
- **Evasion Loop** (`evasion_loop.py`): orchestra l’intero ciclo di Trial & Error.

Il flusso dati che lega questi componenti può essere riassunto come segue: il Traffic Analyzer produce un profilo di baseline; l’Evasion Loop invia un primo pacchetto deliberatamente anomalo al target; il Success Feedback Analyzer classifica la risposta ricevuta (accettazione, scarto silenzioso, rifiuto esplicito); l’LLM Engine, ricevendo sia la baseline sia lo storico dei tentativi falliti, propone la mutazione successiva; il Mutation Engine la applica concretamente al pacchetto; il ciclo si ripete fino al successo o al raggiungimento di un numero massimo di iterazioni.

3.3 Ambiente Dockerizzato e isolamento di rete

L'intera infrastruttura è containerizzata tramite Docker Compose, con due immagini distinte collegate sulla stessa rete Docker isolata. Il container `attacker_client`, costruito a partire da `Dockerfile.attacker`, dispone dei privilegi `NET_ADMIN` necessari a costruire e inviare pacchetti raw tramite Scapy e ospita l'intero codice presente in `src/`. Il container `target_server`, costruito a partire da `Dockerfile.target`, espone un server Nginx protetto da un insieme di regole Iptables applicate automaticamente all'avvio tramite lo script `setup_defense.sh`. L'isolamento a livello di rete Docker garantisce che l'intero ciclo di attacco e difesa avvenga in un ambiente controllato, senza alcun impatto sulla macchina host né su reti esterne, un requisito imprescindibile per un progetto di questo tipo, discusso più diffusamente dal punto di vista etico nel Capitolo 6.

3.4 Traffic Analyzer: profilazione della baseline

Il compito del Traffic Analyzer è costruire un profilo statistico del traffico di rete considerato "normale", da fornire come contesto all'LLM affinché le mutazioni proposte restino plausibili agli occhi del firewall. Operando come sniffer passivo, il modulo osserva il traffico diretto al `target_server` (ad esempio generato inviando ripetutamente richieste `curl`) ed estrae le caratteristiche salienti dei pacchetti: dimensione media, valori tipici di campi di intestazione quali TTL e window size e altre metriche strutturali. Il risultato viene serializzato in `baseline.json`. Per garantire la possibilità di avviare immediatamente un test anche in assenza di traffico reale precedentemente catturato, il repository [15] fornisce un file `baseline_example.json` già pronto, da copiare come punto di partenza.

3.5 Mutation Engine: mutazione dei pacchetti tramite Scapy

Il Mutation Engine espone un insieme di funzioni, costruite sopra la libreria Scapy [4], ciascuna responsabile dell'alterazione di uno specifico campo del pacchetto

raw: i flag TCP, il TTL del livello IP, la window size e la porta sorgente. Ogni funzione riceve in input un pacchetto Scapy e restituisce una nuova istanza mutata, senza alterare lo stato del pacchetto originale, una scelta che consente all'orchestratore di mantenere una cronologia coerente dei tentativi effettuati. Coerentemente con il vincolo di coerenza funzionale discusso nel Capitolo 2, le mutazioni disponibili sono vincolate a non compromettere la struttura minima necessaria affinché un pacchetto TCP resti sintatticamente valido.

3.6 LLM Engine: motore reale e motore Mock

Il cuore decisionale del sistema è `llm_engine.py`, che interroga le API di Google Gemini [5] a ogni iterazione del ciclo. Il prompt inviato al modello combina tre elementi: il profilo di baseline, la cronologia dei tentativi falliti (letta da `evasion_log.json`) e un'istruzione esplicita di restituire un output strutturato in formato JSON, contenente sia il ragionamento che ha condotto alla scelta (in linea con il paradigma Chain-of-Thought [8]) sia le istruzioni di mutazione da applicare. Questa struttura di output rigorosa è motivata da un problema pratico osservato empiricamente durante lo sviluppo: un LLM generalista, se non vincolato a un formato di risposta prevedibile, tende a introdurre variazioni nella sintassi dell'output che ne rendono difficile il parsing automatico da parte dell'orchestratore. Il modulo implementa inoltre una gestione esplicita delle allucinazioni del modello (ad esempio la proposta di una mutazione non prevista dal Mutation Engine o di un formato di output non conforme), scartando l'output non valido, oltre a un meccanismo di rate limiting strutturato per non sovraccaricare né il server bersaglio né l'infrastruttura API sottostante.

La scelta del modello Gemini specifico da utilizzare non è stata immediata, ma è il risultato di una fase di sperimentazione comparativa tra tre modelli della famiglia Gemini 3.5, distinti per capacità di ragionamento e livello di restrizioni di sicurezza applicate lato API. Il modello Gemini 3.5 Pro, pur essendo il più capace in termini di ragionamento, rifiutava sistematicamente di generare una risposta, qualificando esplicitamente la richiesta come un tentativo di elusione di un dispositivo di sicurezza di rete. Il modello Gemini 3.5 Flash, pur non rifiutando la richiesta nel

merito, non rispettava in modo affidabile il formato di output strutturato in JSON richiesto dal sistema, restituendo invece testo libero non parsabile automaticamente. Il modello infine adottato, Gemini 3.5 Flash-Lite, si è dimostrato l'unico a soddisfare simultaneamente entrambi i requisiti, accettare la natura del task e rispettare rigorosamente il formato di output, sebbene a fronte di una capacità di ragionamento inferiore, che si traduce in un numero maggiore di iterazioni necessarie a individuare una mutazione efficace rispetto a quanto ci si potrebbe attendere da un modello più capace. Questo compromesso tra capacità del modello e restrizioni di sicurezza applicate viene discusso ampiamente, insieme alle sue implicazioni metodologiche, nel Capitolo 5.

Accanto al motore reale, `llm_engine_mock.py` implementa un'interfaccia equivalente che restituisce mutazioni predefinite e pseudo-casuali senza effettuare alcuna chiamata di rete. L'architettura a interfacce intercambiabili, motore reale e motore Mock condividono la stessa firma di funzione, permettendo la loro sostituzione trasparente da parte dell'orchestratore che consente di eseguire l'intero ciclo di test in modalità completamente offline, sia per finalità di debug sia per isolare sperimentalmente il contributo specifico del ragionamento dell'LLM rispetto al resto della pipeline, approfondito nella metodologia sperimentale del Capitolo 4.

3.7 Success Feedback Analyzer e orchestratore

Il `successFeedbackAnalyzer.py` ha il compito di interpretare la risposta del target dopo l'invio di ciascun pacchetto mutato, distinguendo tra un pacchetto accettato, uno scartato silenziosamente (DROP) e uno esplicitamente rifiutato (REJECT, tipicamente segnalato da un pacchetto TCP RST), come introdotto nel Capitolo 2. L'esito di questa valutazione viene infine tradotto in un punteggio (+1 in caso di successo, -1 in caso di fallimento) che, insieme ai dettagli del tentativo, viene accodato a `evasion_log.json`.

L'intero ciclo è coordinato da `evasion_loop.py`, l'orchestratore principale, che implementa la seguente sequenza logica: (1) generazione di un pacchetto iniziale deliberatamente anomalo (ad esempio un TCP XMAS scan, secondo la tecnica descritta nel Capitolo 2); (2) invio del pacchetto al `target_server`; (3) valutazione

della risposta tramite il Success Feedback Analyzer; (4) in caso di fallimento, interrogazione dell'LLM Engine per ottenere la mutazione successiva e applicazione della stessa tramite il Mutation Engine; (5) ripetizione del ciclo fino al successo o al raggiungimento di un numero massimo configurabile di iterazioni. Questa struttura, per quanto concettualmente semplice, è ciò che permette al sistema di replicare un processo cognitivo di osservazione, ipotesi, verifica e correzione tipicamente riservato all'analista umano.

Metodologia Sperimentale

4.1 Setup del testbed

L'intera sperimentazione è stata condotta su un personal computer di fascia consumer, all'interno dell'ambiente Dockerizzato descritto nel Capitolo 3: due container collegati sulla stessa rete Docker isolata, senza alcuna esposizione verso reti esterne. Questa scelta, per quanto non equivalente a un'infrastruttura di test dedicata, è coerente con l'obiettivo della tesina, che non mira a valutare prestazioni in termini di throughput o latenza di rete, bensì la capacità dell'agente di ragionare correttamente sulla base delle risposte ricevute, un aspetto non sensibile alle caratteristiche hardware della macchina ospitante.

Il dispositivo di difesa target è stato configurato tramite lo script `setup_defense.sh`, eseguito automaticamente all'avvio del container `target_server`. Il Listato 4.1 riporta la configurazione integrale utilizzata in tutti gli esperimenti.

Listing 4.1: Configurazione delle regole Iptables sul target (`setup_defense.sh`)

```
#!/bin/bash
# Setup basic defensive iptables rules for the target_server
```

```
echo "Setting_up_defensive_firewall_rules..."
```

```
# Logga e scarta pacchetti XMAS (FIN, PSH, URG)
iptables -A INPUT -p tcp --tcp-flags FIN,SYN,RST,PSH,ACK,URG FIN,
    PSH,URG -j LOG --log-prefix "FW-DROP-XMAS:_"
iptables -A INPUT -p tcp --tcp-flags FIN,SYN,RST,PSH,ACK,URG FIN,
    PSH,URG -j DROP

# Logga e scarta i pacchetti con valori TTL insoliti (es. 111)
iptables -A INPUT -m ttl --ttl-eq 111 -j LOG --log-prefix "FW-DROP-
    TTL111:_" || true
iptables -A INPUT -m ttl --ttl-eq 111 -j DROP || true

# Logga e scarta il traffico che finge di essere DNS (porta
    sorgente 53)
# e che tenta di connettersi al server web (porta destinazione 80)
iptables -A INPUT -p tcp --sport 53 --dport 80 -j LOG --log-prefix
    "FW-DROP-BADPORT:_"
iptables -A INPUT -p tcp --sport 53 --dport 80 -j DROP
```

```
echo "Defensive_firewall_rules_configured."
```

Il ruleset adottato non è banale: combina tre euristiche difensive di natura diversa. La prima regola intercetta la classica anomalia *TCP XMAS* (flag FIN, PSH e URG attivi simultaneamente), un pattern che non ha corrispondenza in alcun traffico TCP legittimo e rappresenta quindi un bersaglio “facile” per un sistema di detection basato su firma. La seconda regola scarta pacchetti con un valore di TTL fissato e anomalo (111), simulando un controllo basato sulla deviazione da un profilo di traffico atteso. La terza regola intercetta un tentativo di spoofing della porta sorgente (53, tipica del DNS) verso la porta 80, una tecnica storicamente utilizzata per eludere filtri stateless poco accurati che tendono a fidarsi di traffico apparentemente proveniente da un servizio DNS legittimo. La scelta di combinare tre meccanismi di rilevamento eterogenei, anziché una singola regola isolata, non è casuale: un ruleset a regola singola avrebbe reso banale l’individuazione della mutazione corretta anche per una

strategia di ricerca non informata, vanificando l’obiettivo di verificare se l’agente fosse in grado di un ragionamento realmente adattivo.

È opportuno segnalare fin da ora un limite dello studio, che verrà ripreso nel Capitolo 5: la sperimentazione si è concentrata su questa singola configurazione del firewall, privilegiando un’analisi qualitativa approfondita del comportamento dell’agente su uno scenario controllato piuttosto che una copertura ampia ma superficiale di configurazioni multiple.

4.2 Definizione degli scenari di test

La sperimentazione è stata strutturata attorno a un confronto tra due motori di orchestrazione: il motore Mock e il motore LLM reale, a loro volta articolati in configurazioni successive, riassunte in Tabella 4.1.

Tabella 4.1: Scenari sperimentali definiti

Scenario	Motore	Selezione mutazione	Max iter.
S0	Mock	Casuale (5 mutazioni predefinite)	10
S1	LLM reale (prompt iniziale)	Ragionamento LLM (senza vincolo esplicito)	5, poi 10
S2	LLM reale (prompt raffinato)	Ragionamento LLM (con istruzione di cambio strategia)	10
S3	LLM reale (S2, esecuzione a caldo)	Ragionamento LLM + log storico non vuoto	10

Scenario S0 (Mock). Il motore `llm_engine_mock.py` seleziona a ogni iterazione, in modo casuale e uniforme, una mutazione tra cinque predefinite: alterazione dei flag TCP (due varianti), del TTL, della window size e della porta sorgente. Esattamente una di queste cinque mutazioni corrisponde a una configurazione capace di eludere il ruleset descritto in Sezione 4.1. Questo scenario non ha l’obiettivo di valutare una strategia “intelligente”, bensì di svolgere una duplice funzione metodologica:

verificare la correttezza end-to-end della pipeline di orchestrazione indipendentemente dalla qualità del ragionamento e fornire un termine di paragone, una sorta di *ricerca casuale non informata*, rispetto al quale valutare se il motore LLM reale introduca un vantaggio effettivo in termini di efficienza della ricerca.

Scenario S1 (LLM reale, prompt iniziale). Nella sua prima formulazione, il prompt forniva al modello la baseline di traffico e lo storico dei tentativi falliti, chiedendo di proporre una nuova mutazione, senza però alcuna indicazione esplicita su *come* valutare l’efficacia della strategia adottata fino a quel momento. Il numero massimo di iterazioni è stato inizialmente fissato a 5, poi esteso a 10 nel tentativo di dare al modello margine sufficiente per esplorare lo spazio delle mutazioni disponibili.

Scenario S2 (LLM reale, prompt raffinato). A seguito dell’osservazione empirica del comportamento del modello nello Scenario S1 (approfondita in Sezione 4.3 e discussa nel Capitolo 5), il prompt di sistema è stato modificato per includere un’istruzione esplicita: qualora la strategia di mutazione in corso non stia producendo un miglioramento nel punteggio ricevuto, il modello deve esplicitamente valutare un cambio di approccio anziché insistere su variazioni incrementali dello stesso tipo di campo. Questo scenario è stato eseguito ripristinando un file `evasion_log.json` vuoto (*cold start*), in modo da isolare l’effetto della sola modifica del prompt, senza alcun vantaggio derivante da tentativi precedenti memorizzati su disco.

Scenario S3 (LLM reale, esecuzione a caldo). Una seconda esecuzione dello Scenario S2 è stata condotta senza svuotare `evasion_log.json`, lasciando quindi visibile all’LLM la cronologia della sessione precedente, inclusa la mutazione risultata vincente. Questo scenario permette di distinguere la capacità di *ricerca esplorativa* del modello (necessaria in uno scenario cold-start) dalla sua capacità di *riutilizzo di una strategia nota* quando l’informazione è già disponibile nel contesto, una distinzione rilevante poiché, come discusso nel Capitolo 3, il sistema non implementa alcun meccanismo di apprendimento nei pesi del modello: qualunque forma di “memoria” tra sessioni è interamente affidata alla persistenza del file di log.

4.3 Metriche di valutazione

Per ciascuno scenario sono state raccolte le seguenti misure:

- **Successo del ciclo** (variabile binaria): se il sistema raggiunge un punteggio di +1 (bypass riuscito) entro il numero massimo di iterazioni configurato;
- **Iterazioni-al-successo**: il numero di tentativi effettuati prima del raggiungimento del successo, definito solo per le esecuzioni in cui tale successo si verifica;
- **Traccia di ragionamento**: il contenuto testuale del campo `reasoning` restituito dal modello a ogni iterazione, analizzato qualitativamente per caratterizzare la strategia di esplorazione adottata; in particolare, se il modello tende a concentrarsi su variazioni incrementali di parametri numerici (TTL, window size, porta) oppure su alterazioni categoriche dei flag TCP e se e quando avviene un cambio di strategia;
- **Coerenza tra esecuzioni successive**: confrontando gli Scenari S2 e S3, se una strategia risultata vincente in una sessione precedente viene effettivamente riutilizzata con successo in una sessione successiva, come proxy dell'utilità pratica del meccanismo di logging persistente.

Il confronto primario della tesina è quello tra lo Scenario S0 (baseline non informata) e gli Scenari S1/S2 (ragionamento LLM), volto a rispondere alla domanda di ricerca centrale del lavoro: un agente LLM, opportunamente istruito, converge verso una strategia di evasione efficace in un numero di tentativi comparabile o inferiore rispetto a una ricerca casuale sullo stesso spazio di mutazioni?

4.4 Criteri di riproducibilità

Ai fini della riproducibilità, vanno segnalati alcuni limiti intrinseci alla sperimentazione condotta. In primo luogo, l'utilizzo di un'API commerciale a modello chiuso (Google Gemini) comporta un vincolo di riproducibilità non presente quando si utilizzano modelli open-source con pesi congelati e eseguibili offline: le risposte del modello non sono garantite deterministiche anche a parità di prompt e il comportamento del servizio può variare nel tempo per effetto di aggiornamenti lato provider non controllabili dallo sperimentatore. Per questo motivo, i risultati riportati nel

Capitolo 5 vanno interpretati come osservazioni puntuali relative alle esecuzioni effettivamente condotte, non come garanzie di comportamento riproducibile in senso stretto.

In secondo luogo, la containerizzazione tramite Docker Compose garantisce comunque un livello di riproducibilità dell'ambiente di rete e della configurazione del firewall indipendente dalla macchina host e il repository fornisce sia una baseline di traffico predefinita (`baseline_example.json`) sia un esempio della struttura del log di evasione (`evasion_log_example.json`), a garanzia che un soggetto terzo possa replicare le condizioni di partenza esatte degli Scenari S0, S1 e S2 (cold-start). La replica esatta dello Scenario S3 richiede invece la disponibilità del file di log prodotto da una sessione S2 precedentemente completata con successo, poiché il suo contenuto non è deterministico a priori.

Infine, il numero massimo di iterazioni per ciclo (parametro configurabile dell'orchestratore, impostato a 5 e successivamente a 10 nello Scenario S1 e fissato a 10 negli Scenari S2/S3) e l'intervallo di attesa tra un tentativo e il successivo (fissato a 2 secondi, funzionale al meccanismo di rate limiting discusso nel Capitolo 3) sono parametri espliciti dell'orchestratore e sono riportati qui per consentirne la replica esatta.

Risultati e Analisi

5.1 Risultati sperimentali

La Tabella 5.1 riassume l'esito delle esecuzioni condotte per ciascuno degli scenari definiti nel Capitolo 4.

Scenario S0. Il motore Mock ha raggiunto il successo al quarto tentativo, un risultato coerente con la teoria: con cinque mutazioni equiprobabili di cui una sola efficace, il numero atteso di tentativi prima del successo è pari a 5 e le esecuzioni ripetute si sono attestate quasi sempre entro le prime cinque iterazioni. Questo scenario ha confermato la correttezza end-to-end della pipeline di orchestrazione; invio del pacchetto, valutazione della risposta, applicazione della mutazione, aggiornamento del log, indipendentemente da qualunque capacità di ragionamento, fungendo da termine di paragone per gli scenari successivi.

Scenario S1. Con il prompt nella sua formulazione iniziale, il sistema non è mai giunto a un bypass riuscito, né entro un limite di 5 iterazioni né, successivamente, entro un limite esteso a 10. Il numero complessivo di tentativi effettuati sommando le esecuzioni ripetute ha superato le 40–50 mutazioni proposte dal modello, tutte fallite. L'analisi qualitativa delle tracce di ragionamento raccolte in questa fase (approfondita in Sezione 5.2) ha rivelato la causa del fallimento: il modello tendeva a fissarsi su

variazioni incrementali di parametri numerici (valori di TTL, window size, numero di porta), senza mai esplorare in modo sistematico un’alterazione dei flag TCP diversa da quella tentata inizialmente, pur essendo quest’ultima l’unica classe di mutazione effettivamente efficace contro il ruleset configurato.

Scenario S2. A seguito della modifica del prompt di sistema, l’aggiunta dell’istruzione esplicita di cambiare strategia quando quella corrente non produce miglioramenti nel punteggio, il sistema ha raggiunto il successo al sesto tentativo proposto dal modello (settimo se si conta anche il pacchetto XMAS iniziale privo di mutazioni), a partire da un file `evasion_log.json` vuoto. La sequenza di mutazioni tentate, ricostruibile dal log, è stata:

- alterazione dei flag TCP a PA;
- alterazione del TTL a 64;
- alterazione dei flag a A;
- alterazione della window size a 1024;
- alterazione dei flag a FA;
- alterazione dei flag a S (flag SYN), quest’ultima risultata vincente.

Scenario S3. Ripetendo l’esecuzione senza azzerare il log prodotto dallo Scenario S2, il sistema ha raggiunto il successo al primo tentativo. La traccia di ragionamento raccolta in questa esecuzione mostra esplicitamente il modello fare riferimento alla cronologia registrata nel log, anziché ripartire da un’esplorazione priva di contesto: *“Dalla cronologia dei test, la mutazione dei flag TCP a ‘S’ ha ottenuto successo nel superare il filtro rispetto ad altre strategie come finestra TCP o TTL.”* Il modello ha quindi riproposto direttamente tale mutazione, ottenendo un esito positivo (Score +1) al primo pacchetto inviato. Questo risultato, per quanto ottenuto da un’unica esecuzione e dunque non generalizzabile in senso statistico, fornisce un’evidenza diretta, non solo inferita dal dato numerico, ma osservabile nel testo stesso del ragionamento prodotto dal modello, del ruolo della persistenza del log: la difficoltà del sistema non risiede nel riconoscere una strategia vincente una volta osservata, ma nell’esplorarla

efficacemente lo spazio delle mutazioni quando nessuna informazione pregressa è disponibile.

Tabella 5.1: Sintesi dei risultati sperimentali per scenario

Scenario	Esito	Iter. al successo	Note
S0 (Mock)	Successo	4	Selezione casuale su 5 mutazioni predefinite
S1 (LLM, prompt iniziale)	Fallimento	—	Oltre 40–50 tentativi complessivi, nessun successo
S2 (LLM, prompt rafinato, cold start)	Successo	6	Log azzerato prima dell'esecuzione
S3 (LLM, prompt rafinato, warm start)	Successo	1	Log della sessione S2 non azzerato

5.2 Analisi delle strategie di evasione scoperte dall'agente

Le tracce di ragionamento restituite dal modello nello Scenario S2 permettono di ricostruire, passo per passo, il processo di esplorazione seguito. Dopo il fallimento del pacchetto XMAS iniziale, il modello ha correttamente osservato che la baseline di traffico mostrava una prevalenza di pacchetti con flag `ACK` e `PSH-ACK` e ha proposto di conseguenza una mutazione verso il flag `PA`, un'inferenza plausibile, mimetizzare il pacchetto rispetto al traffico osservato, ma rivelatasi inefficace contro il ruleset reale, che non filtra affatto sulla base di questi flag specifici. Falliti anche i tentativi successivi sul `TTL` e sulla `window size`, motivati da un ragionamento analogo (allineare il pacchetto ai valori medi della baseline), il modello ha infine provato la mutazione del flag `TCP a S (SYN)`, esplicitamente motivata nel log come tentativo di

“testare l’handshake iniziale”, la mutazione realmente efficace, poiché nessuna delle tre regole del ruleset (Sezione 4.1) intercetta un normale pacchetto SYN.

È interessante notare che tale conclusione non è stata raggiunta per deduzione diretta dalla struttura delle regole del firewall, informazione non accessibile al modello, che osserva soltanto l’esito dei propri tentativi e la baseline di traffico legittimo, bensì per esclusione progressiva delle alternative esplorate, un processo che nella sua forma resta coerente con il paradigma Chain-of-Thought [8], sebbene nello Scenario S1 lo stesso paradigma applicato senza l’istruzione di cambio-strategia esplicita si sia rivelato inefficace, rimanendo intrappolato in un’unica famiglia di mutazioni (i parametri numerici) senza mai considerare l’alterazione categorica dei flag come alternativa. Questo suggerisce che la capacità di Chain-of-Thought del modello, di per sé, non garantisca un’esplorazione efficiente dello spazio delle soluzioni e che un’istruzione esplicita orientata a favorire la diversificazione della strategia, un intervento minimo lato prompt engineering, sia stata determinante per il successo, più della capacità di ragionamento in sé.

5.3 Limiti riscontrati

5.3.1 Restrizioni di sicurezza a livello di modello e loro impatto sul sistema

Un limite rilevante, emerso durante lo sviluppo e non immediatamente prevedibile in fase di progettazione, riguarda il comportamento differenziato dei modelli Gemini rispetto a richieste riconducibili al dominio della sicurezza offensiva. È stato osservato che il livello di restrizione applicato non è uniforme all’interno della stessa famiglia di modelli, ma sembra correlato al livello di capacità del modello stesso: il modello più capace testato (Gemini 3.5 Pro) ha rifiutato esplicitamente e sistematicamente qualunque richiesta riconducibile all’elusione di un dispositivo di sicurezza di rete, indipendentemente dalla formulazione del prompt. Un modello di capacità intermedia (Gemini 3.5 Flash) non presentava un rifiuto esplicito a livello di contenuto, ma applicava una restrizione più sottile, impedendo la generazione di output in formato strutturato (JSON) qualora il contenuto della risposta fosse riconducibile a

istruzioni potenzialmente azionabili da un programma esterno, un comportamento verosimilmente riconducibile a un meccanismo di sicurezza orientato specificamente a impedire l'integrazione automatica dell'output in pipeline software. Solamente il modello più leggero della famiglia (Gemini 3.5 Flash-Lite) si è dimostrato utilizzabile per gli scopi del progetto, accettando la natura simulata e isolata del task se opportunamente contestualizzata nel prompt di sistema e rispettando in modo affidabile il formato di output richiesto.

Questa osservazione ha due implicazioni degne di nota. La prima è di natura pratica: è stato necessario un lavoro non trascurabile di *prompt engineering* per ottenere un formato di risposta utilizzabile dall'orchestratore, in particolare esplicitando nel prompt di sistema il contesto isolato, controllato e a fini di ricerca accademica in cui il sistema opera; un chiarimento che di per sé non era sufficiente sui modelli a capacità più elevata, ma che si è rivelato risolutivo sul modello Flash-Lite. La seconda implicazione è più metodologica: il compromesso tra capacità di ragionamento e severità dei filtri di sicurezza applicati introduce un trade-off non banale nella progettazione di sistemi di questo tipo, poiché il modello più adatto all'integrazione in una pipeline automatizzata (Flash-Lite) è al contempo quello con minore capacità di ragionamento strategico, richiedendo empiricamente un numero maggiore di iterazioni per individuare una mutazione efficace rispetto a quanto teoricamente atteso da un modello più capace. Va inoltre segnalato che, una volta individuata con successo una mutazione efficace per una determinata configurazione del firewall, il sistema tendeva a replicare tale scelta con successo al primo tentativo nelle esecuzioni successive, suggerendo che il collo di bottiglia risieda più nella fase di esplorazione iniziale che nella capacità di generalizzazione del modello, come confermato dal confronto diretto tra gli Scenari S2 e S3 in Sezione 5.1.

5.3.2 Altri limiti dello studio

Coerentemente con lo scope della tesina: realizzazione di un Proof of Concept mirato a verificare un'ipotesi specifica, non una validazione esaustiva, la sperimentazione presenta due limiti che è opportuno dichiarare esplicitamente, entrambi naturali candidati per un lavoro di approfondimento futuro (Capitolo 7).

In primo luogo, come già anticipato nel Capitolo 4, gli esperimenti sono stati condotti contro un'unica configurazione del firewall target. Sebbene tale configurazione combini euristiche di natura eterogenea (anomalia di flag, deviazione dal TTL atteso, spoofing di porta) proprio per evitare di ridurre il problema a un caso banale, resta da verificare se il comportamento osservato dell'agente, in particolare la sua tendenza iniziale a esplorare mutazioni numeriche prima di considerare i flag TCP, si mantenga anche contro ruleset di natura sostanzialmente diversa, ad esempio basati su ispezione a livello applicativo anziché di trasporto.

In secondo luogo, l'analisi qui presentata ha natura prevalentemente qualitativa: il numero di esecuzioni documentate per scenario riflette la scelta di privilegiare una ricostruzione approfondita del ragionamento dell'agente, passo per passo, piuttosto che un numero elevato di ripetizioni finalizzate a stime statistiche aggregate (media e varianza delle iterazioni al successo). Questa scelta è coerente con la domanda di ricerca posta nell'Introduzione, se e come un LLM ragioni in un ciclo di evasione, ma implica che i risultati numerici riportati in Tabella 5.1 vadano letti come osservazioni puntuali illustrative del fenomeno, non come stime consolidate della sua distribuzione statistica.

5.4 Discussione critica dei risultati

Il confronto tra gli scenari sperimentali permette di formulare alcune osservazioni conclusive. Il dato più significativo emerso è che la capacità del modello linguistico di per sé non è risultata sufficiente a garantire una convergenza efficiente verso una strategia di evasione: lo Scenario S1, pur utilizzando lo stesso modello (Gemini 3.5 Flash-Lite) e lo stesso spazio di mutazioni disponibili dello Scenario S2, non ha mai raggiunto il successo in oltre 40–50 tentativi, mentre una singola modifica mirata del prompt, ha permesso al sistema di convergere in appena 6 tentativi. Questo risultato indica che, almeno nel dominio circoscritto qui esaminato, l'efficacia di un agente LLM applicato all'evasione di un firewall dipende in misura significativa dalla qualità dell'istruzione di meta-strategia fornita nel prompt, più che dalla capacità di ragionamento intrinseca del modello sottostante, un'osservazione che ridimensiona, almeno in questo contesto specifico, l'idea di un agente pienamente autonomo capace

di ragionamento strategico emergente senza guida esplicita e che risulta coerente con le criticità più generali sollevate da Happe [3] circa l’affidabilità pratica degli LLM in compiti di sicurezza offensiva.

Il confronto con lo Scenario S0 (Mock) va letto con la giusta prospettiva metodologica. Con uno spazio di sole cinque mutazioni disponibili, di cui una sola efficace, il numero di tentativi necessari a una ricerca casuale non informata è per costruzione basso (5 in media) e non stupisce quindi che lo Scenario S2 (6 tentativi) si collochi in un ordine di grandezza simile. Questo dato numerico, tuttavia, coglie solo una parte del confronto: a differenza del motore Mock, che seleziona una mutazione a caso senza alcuna relazione con l’esito dei tentativi precedenti, la sequenza di mutazioni osservata nello Scenario S2 mostra un’esplorazione motivata, in cui ogni scelta è giustificata esplicitamente sulla base del punteggio ricevuto e delle caratteristiche della baseline, un comportamento che il numero di iterazioni da solo non cattura, ma che è visibile nel contenuto stesso del ragionamento prodotto dal modello (Sezione 5.2). Inoltre, il confronto tra gli Scenari S2 e S3 evidenzia una capacità che il motore Mock non possiede per costruzione, ovvero il riutilizzo efficace di un’informazione pregressa: mentre il Mock ripeterebbe una selezione casuale indipendentemente dalla storia, l’agente LLM ha sfruttato la cronologia registrata nel log per risolvere il problema al primo tentativo.

Ciò detto, per quantificare in modo più conclusivo il vantaggio di un agente LLM rispetto a una ricerca non informata, distinguendo chiaramente un vantaggio in termini di efficienza pura da un vantaggio in termini di qualità e trasferibilità del ragionamento, qui osservato solo qualitativamente, sarebbe utile uno spazio di mutazioni sensibilmente più ampio e un ruleset target di complessità maggiore, condizioni che rappresentano una direzione naturale per un lavoro di approfondimento futuro, discussa nel Capitolo 7.

Considerazioni Etiche e Implicazioni di Sicurezza

6.1 Uso responsabile della ricerca offensiva

Un progetto che ha per oggetto l'automazione di tecniche di elusione dei controlli di rete solleva, per sua stessa natura, interrogativi etici che non possono essere ignorati né liquidati con una semplice dichiarazione di intenti. Il quadro di riferimento più consolidato in letteratura per la ricerca in sicurezza informatica è il Menlo Report [16], che estende al dominio ICT i principi del rispetto per le persone, della beneficenza e della giustizia già alla base dell'etica della ricerca biomedica. Applicato al presente lavoro, questo quadro si traduce in due garanzie progettuali concrete, già descritte dal punto di vista tecnico nel Capitolo 3: l'assenza di qualsiasi soggetto terzo potenzialmente danneggiato e il contenimento totale del rischio all'interno di un ambiente controllato dallo sperimentatore stesso.

Nello specifico, l'intero sistema opera esclusivamente contro un bersaglio costruito ad-hoc (il container `target_server`), di cui lo sperimentatore è al contempo proprietario e amministratore, all'interno di una rete Docker isolata priva di qualunque instradamento verso l'esterno. Non è stato in alcun momento testato, né si è avuta l'intenzione di testare, un sistema di terzi privo di autorizzazione esplicita: una distinzione che nella prassi della sicurezza offensiva professionale corrisponde

esattamente al confine tra penetration testing autorizzato e attività illecita. In secondo luogo, il progetto è stato condotto con finalità dichiaratamente accademiche, nell'ambito di un percorso di studio in Penetration Testing and Ethical Hacking e non è stato distribuito né promosso come strumento pronto all'uso al di fuori di tale contesto didattico.

6.2 Implicazioni difensive

Sebbene il sistema sviluppato assuma il punto di vista dell'attaccante, il suo valore principale, in una prospettiva più ampia, risiede nelle implicazioni per la difesa. Un sistema capace di individuare autonomamente, tramite ragionamento iterativo, quale caratteristica di un pacchetto risulti effettivamente ispezionata da un firewall costituisce, di fatto, uno strumento di *auditing automatico* della postura difensiva di un sistema di controllo di rete: la stessa logica che nel Capitolo 5 ha permesso all'agente di individuare che il ruleset configurato non filtrava affatto sui flag TCP standard rappresenta un'informazione preziosa anche per chi quel firewall lo deve configurare e mantenere.

In questo senso, un sistema di questo tipo può essere impiegato in ottica di red teaming continuo: anziché affidarsi a un singolo audit periodico condotto da un professionista umano, un'infrastruttura potrebbe sottoporre le proprie regole di filtraggio a un ciclo automatizzato di stress-test simile a quello qui descritto, individuando lacune di configurazione prima che vengano scoperte e sfruttate da un attaccante reale. Questa prospettiva è coerente con un principio ormai consolidato nella pratica della sicurezza informatica, secondo cui comprendere a fondo le tecniche offensive costituisce un prerequisito, non un'alternativa, alla costruzione di difese efficaci.

6.3 Rischi di dual-use e mitigazioni

Il rischio di uso duale di questo lavoro è reale e non va minimizzato: le medesime tecniche di mutazione dei pacchetti descritte nel Capitolo 3, se applicate contro un sistema non autorizzato, costituirebbero un'attività illecita a tutti gli effetti. Un

possibile dubbio etico riguarda la pubblicazione stessa di queste informazioni: descrivere tecniche di elusione di un firewall potrebbe sembrare pericoloso. Va però osservato che tali tecniche non sono una scoperta di questa tesina, ma sono note e pubblicamente documentate in letteratura scientifica da oltre venticinque anni [1]. Il contributo specifico di questo lavoro, l'automazione del ragionamento tramite un agente LLM, introduce semmai una considerazione aggiuntiva, relativa non tanto alla tecnica in sé quanto alla sua accessibilità: un processo che richiedeva competenza specialistica ed esperienza diretta può, in linea di principio, essere reso disponibile a un numero maggiore di soggetti attraverso un'interfaccia in linguaggio naturale. Le mitigazioni adottate in questo lavoro sono di natura sia tecnica sia procedurale: il codice sorgente non è stato distribuito come strumento pronto all'uso contro bersagli arbitrari, la validità del sistema è stata dimostrata esclusivamente in un ambiente isolato e autocontenuto e la presente tesina descrive il funzionamento del sistema a un livello di dettaglio sufficiente alla comprensione accademica del fenomeno senza costituire una guida operativa passo-passo per un utilizzo malevolo.

Un aspetto ulteriore, emerso nel corso dello sviluppo e meritevole di discussione esplicita, riguarda la relazione tra questo lavoro e le misure di sicurezza applicate direttamente dal fornitore del modello linguistico utilizzato. Come descritto nel Capitolo 3 (Sezione 5.3.1), i modelli Gemini più capaci si sono rifiutati, in modo sistematico e per costruzione, di generare contenuti riconducibili all'elusione di un dispositivo di sicurezza di rete e il completamento del progetto ha richiesto una riformulazione del prompt volta a contestualizzare esplicitamente la natura isolata e accademica della richiesta. Questa circostanza solleva una tensione etica che è onesto riconoscere esplicitamente, piuttosto che considerarla un mero dettaglio implementativo: i filtri di sicurezza applicati da un fornitore di modelli linguistici costituiscono una misura di mitigazione del rischio pensata per operare su larga scala, senza possibilità per il modello di verificare autonomamente la veridicità del contesto dichiarato dall'utente; un ricercatore che, pur con intenti legittimi, formula un prompt specificamente per superare tale misura si colloca in una zona che, se generalizzata a un numero indefinito di utenti con le più svariate intenzioni, indebolirebbe l'efficacia complessiva della misura stessa. Non essendo possibile, dal lato dell'utente, dimostrare in modo verificabile la propria buona fede a un sistema automatizzato, la responsabilità etica

di questa scelta ricade interamente sullo sperimentatore, ed è stata qui gestita mantenendo l'attività rigorosamente confinata a un ambiente isolato, non distribuendo la formulazione esatta del prompt utilizzato e documentando la circostanza in questa tesina con l'obiettivo di renderla oggetto di riflessione esplicita, una trasparenza che, a parere di chi scrive, rappresenta l'unica postura eticamente sostenibile quando si sceglie di muoversi in questo tipo di zona grigia a fini di ricerca accademica.

Conclusioni e Sviluppi Futuri

7.1 Sintesi dei contributi

Questa tesina ha presentato *AI-Driven Firewall Evasion System*, un Proof of Concept che affida a un Large Language Model il ruolo di agente decisionale all'interno di un ciclo iterativo di elusione di un firewall Iptables, sostituendo il ragionamento manuale del penetration tester con un processo automatizzato di osservazione, ipotesi e correzione. Il lavoro ha attraversato l'intero percorso che va dalla motivazione teorica (Capitoli 1 e 2) alla realizzazione tecnica (Capitolo 3), fino alla verifica sperimentale delle ipotesi di partenza (Capitoli 4 e 5).

I risultati raccolti indicano che la sola capacità di ragionamento di un LLM generalista non è, di per sé, sufficiente a garantire una convergenza efficiente verso una strategia di evasione: lo scarto tra lo Scenario S1 e lo Scenario S2 mostra che l'efficacia pratica di un agente di questo tipo dipende in misura significativa dalla qualità dell'istruzione di meta-strategia fornita, più che dalla potenza bruta del modello sottostante. A questo si aggiunge un risultato meno atteso ma altrettanto rilevante: il vincolo più severo incontrato nello sviluppo non è stato di natura tecnica, bensì derivante dalle misure di sicurezza applicate dal fornitore del modello linguistico, che hanno reso necessario un compromesso esplicito tra capacità di ragionamento

del modello e la sua effettiva utilizzabilità all'interno di una pipeline automatizzata, un tema approfondito nel Capitolo 5 e ripreso in chiave etica nel Capitolo 6.

Il progetto, per sua natura di Proof of Concept realizzato nell'ambito del corso di Penetration Testing and Ethical Hacking, non ha la pretesa di fornire uno strumento maturo né una validazione statisticamente esaustiva, obiettivi peraltro incompatibili con i tempi a disposizione per una tesina d'esame. Il suo valore, come discusso nella Sezione 5.3 e nella Sezione 5.4 del Capitolo 5, risiede nell'aver isolato sperimentalmente, tramite un confronto diretto tra un motore Mock e un motore LLM reale, l'effettivo contributo del ragionamento artificiale rispetto a una ricerca non informata e nell'aver documentato con onestà tanto i risultati positivi quanto i limiti e le difficoltà incontrate.

7.2 Sviluppi futuri

Diverse direzioni di approfondimento sono emerse naturalmente nel corso della tesina e costituiscono altrettanti punti di partenza per un lavoro di ricerca più esteso, di cui questo Proof of Concept rappresenta la base sperimentale:

- **Estensione a scenari di detection multipli.** Come segnalato in Sezione 5.3, la sperimentazione qui condotta ha riguardato un'unica configurazione del firewall target. Un naturale sviluppo consiste nel definire una batteria di ruleset di complessità crescente, per verificare se il comportamento esplorativo osservato dell'agente (in particolare la tendenza iniziale a privilegiare mutazioni numeriche rispetto a quelle categoriche) si mantenga costante o vari in funzione della natura del meccanismo di detection.
- **Ritorno all'ampiezza del design originale.** Il documento di design da cui è nato questo progetto (si veda il Capitolo 3) prevedeva mutazioni anche a livello applicativo (offuscamento degli header HTTP e segmentazione irregolare dei record TLS), non implementate nella versione qui realizzata. Estendere il Mutation Engine a questi livelli del protocollo consentirebbe di verificare se le difficoltà di esplorazione osservate a livello di trasporto (Capitolo 5) si

ripresentino, in forma analoga o diversa, anche su uno spazio di mutazioni più ampio ed eterogeneo.

- **Superamento del compromesso tra capacità del modello e restrizioni di sicurezza.** Il trade-off discusso nella Sezione 5.3.1; dover scegliere tra un modello capace ma bloccato dai filtri di sicurezza e un modello meno capace ma utilizzabile, potrebbe essere superato adottando un modello open-source eseguito localmente sul proprio hardware, privo di restrizioni imposte da un fornitore esterno. Questo approccio permetterebbe inoltre di allenare il modello in modo specifico sul dominio della sicurezza di rete tramite fine-tuning (l'addestramento aggiuntivo di un modello già pronto su un insieme di dati mirato), sulla scia di quanto proposto da lavori come xOffense [11], superando al contempo il limite di riproducibilità legato a un'API commerciale a modello chiuso discusso in Sezione 4.4.
- **Validazione statistica su ampia scala.** Come discusso in Sezione 5.3.2, i risultati qui presentati hanno natura prevalentemente qualitativa. Un numero maggiore di esecuzioni ripetute per ciascuno scenario permetterebbe di calcolare metriche aggregate: tempo medio di convergenza, tasso di successo entro un numero fissato di iterazioni e le metriche di *evasion success rate* e *mutation stealth* già previste nel design originale del progetto, fornendo una base statisticamente più solida al confronto tra ricerca guidata da LLM e ricerca non informata discusso in Sezione 5.4.
- **Verso un'architettura multi-agente.** Coerentemente con l'evoluzione osservata in letteratura (Sezione 2.2), un possibile sviluppo consiste nel suddividere le responsabilità oggi concentrate in un singolo LLM Engine tra più agenti specializzati; ad esempio un agente dedicato all'interpretazione della baseline e uno alla formulazione della strategia di mutazione, verificando se una simile suddivisione di ruoli produca un'esplorazione più efficiente dello spazio delle mutazioni rispetto a quanto osservato con un agente monolitico.

Queste direzioni, prese singolarmente o in combinazione, delineano un percorso di lavoro naturale per l'approfondimento di questo Proof of Concept in una tesi di

laurea, di cui il presente progetto costituisce la base sperimentale e concettuale.

7.3 Riflessione conclusiva

Al termine di questo lavoro, vale la pena tornare per un momento alla domanda che lo ha motivato fin dall'Introduzione: un agente artificiale può replicare il ragionamento di un analista umano di fronte a un sistema di difesa che non conosce? La risposta emersa da questa sperimentazione è, come spesso accade nella ricerca, più sfumata di un semplice sì o no. L'agente ha mostrato, nello Scenario S2, un'esplorazione genuinamente motivata piuttosto che casuale; ma ha anche mostrato, nello Scenario S1, quanto tale ragionamento possa arenarsi senza un'indicazione mirata su *come* imparare dai propri fallimenti, un limite che, più che sminuire il valore del sistema, ne chiarisce la reale natura: non un sostituto autonomo dell'analista, ma uno strumento la cui efficacia dipende ancora in modo critico dalla qualità della guida ricevuta.

Questo Proof of Concept nasce come progetto per il corso di Penetration Testing and Ethical Hacking, ma il percorso compiuto per realizzarlo, dalla progettazione iniziale, alla sua necessaria semplificazione in fase implementativa, fino alle difficoltà impreviste emerse nella sperimentazione, prima fra tutte il confronto diretto con i limiti imposti dai meccanismi di sicurezza dei modelli linguistici stessi, ha restituito qualcosa che va oltre la semplice verifica di un'ipotesi tecnica: la conferma che, tanto nell'offesa quanto nella difesa dei sistemi informatici, il valore di uno strumento automatizzato non sta nella sua autonomia assoluta, ma nella qualità del dialogo tra chi lo progetta e il ragionamento che esso produce. È su questa base e con questa consapevolezza, che il presente lavoro intende proseguire nella tesi di laurea.

Bibliografia

- [1] T. H. Ptacek and T. N. Newsham, "Insertion, evasion, and denial of service: Eluding network intrusion detection," Secure Networks, Inc., Tech. Rep., January 1998, foundational technical report on IDS/firewall evasion techniques. (Citato alle pagine 1, 4 e 28)
- [2] G. Deng, Y. Liu, V. Mayoral-Vilches, P. Liu, Y. Li, Y. Xu, T. Zhang, Y. Liu, M. Pinzger, and S. Rass, "Pentestgpt: An llm-empowered automatic penetration testing tool," *arXiv preprint arXiv:2308.06782*, 2023. (Citato alle pagine 1 e 5)
- [3] A. Happe, "On the surprising efficacy of llms for penetration-testing," *arXiv preprint arXiv:2507.00829*, 2025. (Citato alle pagine 1, 5 e 25)
- [4] P. Biondi and The Scapy Community, "Scapy: Packet crafting for python2 and python3," <https://scapy.net>, ultimo accesso: settembre 2026. (Citato alle pagine 2, 5 e 9)
- [5] Google DeepMind, "Gemini api documentation," <https://ai.google.dev/gemini-api/docs>, ultimo accesso: settembre 2026. (Citato alle pagine 2 e 10)
- [6] G. F. Lyon, "Nmap: the network mapper - free security scanner," <https://nmap.org>, ultimo accesso: settembre 2026. (Citato a pagina 4)

-
- [7] D. Song, “fragrouter: a network intrusion detection evasion toolkit,” <https://monkey.org/~dugsong/fragrouter/>, 1999, ultimo accesso: settembre 2026. (Citato a pagina 4)
- [8] J. Wei, X. Wang, D. Schuurmans, M. Bosma, B. Ichter, F. Xia, E. Chi, Q. Le, and D. Zhou, “Chain-of-thought prompting elicits reasoning in large language models,” *arXiv preprint arXiv:2201.11903*, 2022. (Citato alle pagine 5, 8, 10 e 22)
- [9] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, and Y. Cao, “React: Synergizing reasoning and acting in language models,” *arXiv preprint arXiv:2210.03629*, 2022. (Citato a pagina 5)
- [10] H. Kong, D. Hu, J. Ge, L. Li, T. Li, and B. Wu, “Vulnbot: Autonomous penetration testing for a multi-agent collaborative framework,” *arXiv preprint arXiv:2501.13411*, 2025. (Citato a pagina 5)
- [11] P. D. Luong, L. T. G. Bao, N. V. K. Tam, D. H. N. Khoa, N. H. Quyen, V.-H. Pham, and P. T. Duy, “xoffense: An autonomous multi-agent framework for penetration testing with domain-adapted large language models,” *arXiv preprint arXiv:2509.13021*, 2025. (Citato alle pagine 5 e 32)
- [12] E. Alhajjar, P. Maxwell, and N. Bastian, “Adversarial machine learning in network intrusion detection systems,” *Expert Systems with Applications*, vol. 186, p. 115782, 2021. (Citato a pagina 6)
- [13] M. Usama, M. Asim, S. Latif, J. Qadir, and A. Al-Fuqaha, “Generative adversarial networks for launching and thwarting adversarial attacks on network intrusion detection systems,” in *2019 15th International Wireless Communications & Mobile Computing Conference (IWCMC)*. IEEE, 2019, pp. 78–83. (Citato a pagina 6)
- [14] C. Zhang, X. Costa-Pérez, and P. Patras, “Adversarial attacks against deep learning-based network intrusion detection systems and defense mechanisms,” *IEEE/ACM Transactions on Networking*, vol. 30, no. 3, pp. 1294–1311, 2022. (Citato a pagina 6)
- [15] Vittorio Guida, “Ai-driven firewall evasion system,” https://github.com/VittorioGuida15/Firewall_Evasion_Project. (Citato alle pagine 8 e 9)

- [16] D. Dittrich, E. Kenneally, and D. Maughan, "The menlo report," *IEEE Security & Privacy*, vol. 10, no. 2, pp. 71–75, 2012. (Citato a pagina 26)